TikuOS Timer Subsystem

Simple. Ubiquitous. Intelligence, Everywhere.

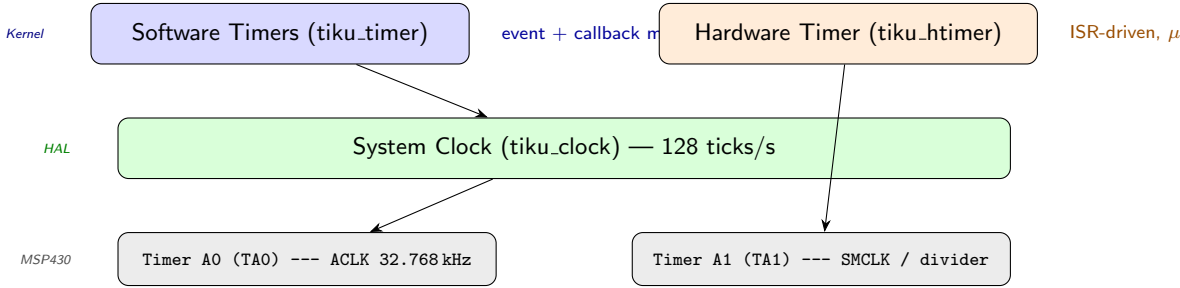
Ambuj Varshney
ambuj@tiku-os.org

March 4, 2026

Outline

1. Timer Architecture
2. System Clock
3. Software Timers
4. Hardware Timer (htimer)
5. Common Patterns
6. File Organization
7. Test Suite
8. Summary

Three-Layer Timer Design



	System Clock	Software Timer	Hardware Timer
Resolution	7.8 ms (128 Hz)	7.8 ms	1 μs (configurable)
Concurrency	—	Multiple (linked list)	Single (one pending)
Context	ISR	Timer process	ISR
Use case	Timekeeping	Process events/callbacks	Precise ISR timing

System Clock Configuration

arch/msp430/tiku_timer_arch.h

```
/* Ticks per second (power of 2) */
#define TIKU_CLOCK_ARCH_SECOND  128

/* ACLK from XT1 crystal */
#define TIKU_CLOCK_ARCH_ACLK_FREQ \
    32768

/* ISR fires every 256 ACLK ticks */
#define TIKU_CLOCK_ARCH_INTERVAL \
    (32768 / 128)  /* = 256 */
```

Clock types

```
typedef unsigned short
    tiku_clock_time_t;       /* 16-bit */
typedef unsigned long
    tiku_clock_arch_time_t; /* 32-bit */
```

How it works:

1. Timer A0 runs from ACLK (32.768 kHz crystal)
2. CCR0 compare interrupt fires every 256 ACLK cycles
3. ISR increments tick counter (128 ticks/s)
4. Every 128 ticks, seconds counter increments
5. ISR wakes CPU from LPM3

TIKU_CLOCK_SECOND

Convenience macro = 128.
One second = 128 ticks.
Half second = `TIKU_CLOCK_SECOND / 2`.

Clock API

Function / Macro	Description
`tiku_clock_init()`	Start Timer A0, enable interrupts
`tiku_clock_time()`	Current tick count (16-bit, wraps)
`tiku_clock_seconds()`	Elapsed seconds since boot
`tiku_clock_wait(t)`	Busy-wait for `t` ticks
`tiku_clock_delay_usec(dt)`	Microsecond-scale busy delay
`TIKU_CLOCK_SECOND`	Ticks per second (128)
`TIKU_CLOCK_MS_TO_TICKS(ms)`	Convert milliseconds to ticks
`TIKU_CLOCK_LT(a,b)`	Wraparound-safe `a < b`
`TIKU_CLOCK_DIFF(a,b)`	Wraparound-safe `a - b`

Wraparound Safety

The 16-bit tick counter wraps every ~512 seconds. **Always** use `TIKU_CLOCK_LT()` and `TIKU_CLOCK_DIFF()` for comparisons — never raw `<` or `-`.

Clock ISR & Atomic Reads

Timer A0 ISR (simplified)

```
TIKU_ISR(TIMER0_A0_VECTOR,
         timer0_a0_isr)
{
    tiku_arch_last_tar = TA0R;
    ++tiku_arch_count;

    if ((tiku_arch_count %
         TIKU_CLOCK_ARCH_SECOND) == 0)
        ++tiku_arch_seconds;

    /* Wake timer process */
    tiku_timer_request_poll();

    /* Exit LPM3 */
    __bic_SR_register_on_exit(
        LPM3_bits);
}
```

Double-read for stability

```
tiku_clock_arch_time_t
tiku_clock_arch_time(void)
{
    tiku_clock_arch_time_t t1, t2;
    do {
        t1 = tiku_arch_count;
        t2 = tiku_arch_count;
    } while (t1 != t2);
    return t1;
}
```

Why Double-Read?

On 16-bit MSP430, a 32-bit read is not atomic. ISR can fire between the two 16-bit loads. Repeat until stable.

Timer Structure & Modes

kernel/timers/tiku_timer.h

```
struct tiku_timer {
    struct tiku_timer *next;
    tiku_clock_time_t  start;
    tiku_clock_time_t  interval;
    uint8_t  mode;     /* EVENT or CB */
    uint8_t  active;
    struct tiku_process *p;
    tiku_timer_callback_t func;
    void *ptr;
};
```

Expiration check:

```
static inline int timer_is_due(
    struct tiku_timer *t,
    tiku_clock_time_t now)
{
    return (tiku_clock_time_t)
        (now - t->start) >= t->interval;
}
```

Two operating modes:

Event Mode

Posts `TIKU_EVENT_TIMER` to the owning process when expired.
Process waits with `WAIT_EVENT_UNTIL(ev == TIMER)`.

Callback Mode

Calls a function directly when expired.
Runs in the timer management process context.
No need to write a process for simple periodic work.

~24 bytes per timer instance.
All statically allocated.

Event Timer API

Set an event timer

```
static struct tiku_timer my_timer;

/* Start a 1-second timer (posts TIKU_EVENT_TIMER to calling process) */
tiku_timer_set_event(&my_timer, TIKU_CLOCK_SECOND);
```

Wait for the timer event

```
TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);
/* Timer has fired --- do work */
```

Periodic: drift-free reset

```
while (1) {
    TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);
    tiku_common_led1_toggle();
    tiku_timer_reset(&my_timer);   /* start += interval (no drift) */
}
```

reset() vs restart()

`tiku_timer_reset()` — advances start by `interval` (drift-free).

Callback Timer API

Set a callback timer

```
static struct tiku_timer cb_timer;

void my_callback(void *ptr) {
    tiku_common_led1_toggle();
    tiku_timer_reset(&cb_timer);   /* reschedule for periodic */
}

/* Start 500ms callback timer */
tiku_timer_set_callback(&cb_timer, TIKU_CLOCK_SECOND / 2,
                        my_callback, NULL);
```

When to Use Callbacks

- Simple periodic actions (LED toggle)
- No need for a full process
- Multiple independent timers
- Self-rescheduling with `reset()`

When to Use Event Timers

- Process already exists
- Need to combine with other events
- Timeout patterns
- Complex state machines

Timer Control Functions

Function	Description
`tiku_timer_set_event(t, ticks)`	Start event timer for calling process
`tiku_timer_set_callback(t, ticks, fn, ptr)`	Start callback timer
`tiku_timer_reset(t)`	Drift-free reschedule: `start += interval`
`tiku_timer_restart(t)`	Reschedule from now: `start = now`
`tiku_timer_stop(t)`	Cancel timer, remove from active list
`tiku_timer_expired(t)`	True if timer is not active
`tiku_timer_remaining(t)`	Ticks until expiration
`tiku_timer_expiration_time(t)`	Absolute tick of expiration
`tiku_timer_any_pending()`	True if any timer is active
`tiku_timer_next_expiration()`	Nearest expiration tick

Timer Management Internals

How software timers are dispatched:

1. Clock ISR fires every 7.8 ms
2. ISR calls `tiku_timer_request_poll()`
3. Timer management process wakes
4. Walks linked list of active timers
5. For each due timer:
   - **Event mode**: post `TIKU_EVENT_TIMER`
   - **Callback mode**: call `func(ptr)`
6. Expired timers removed from list
7. List rescanned after each dispatch (callbacks may modify the list)

Timer process (simplified)

```
TIKU_PROCESS_THREAD(
    tiku_timer_process, ev, data)
{
    TIKU_PROCESS_BEGIN();

    while (1) {
        TIKU_PROCESS_WAIT_EVENT();

        if (ev == TIKU_EVENT_POLL) {
            /* Walk timer list */
            now = tiku_clock_time();
            for each active timer t:
                if timer_is_due(t, now):
                    dispatch(t);
        }
        if (ev == TIKU_EVENT_EXITED)
            /* Cleanup dead process
               timers */
    }

    TIKU_PROCESS_END();
}
```

Hardware Timer: Microsecond Precision

kernel/timers/tiku_htimer.h

```
struct tiku_htimer {
    tiku_htimer_clock_t time;
    tiku_htimer_callback_t func;
    void *ptr;
};

/* Callback signature */
typedef void (*tiku_htimer_callback_t)(
    struct tiku_htimer *t, void *ptr);
```

Status codes

```
TIKU_HTIMER_OK          =  0
TIKU_HTIMER_ERR_TIME    = -1
TIKU_HTIMER_ERR_INVALID = -2
TIKU_HTIMER_ERR_NONE    = -3
```

Key characteristics:

- Uses Timer A1 (separate from clock)
- Default: 1 MHz (1 μs resolution)
- **Singleton**: only one pending at a time
- Callback runs in **ISR context**
- Callback can reschedule itself
- Guard time: ~61 μs minimum

ISR Context

Keep htimer callbacks short.
No blocking, no complex logic.
Just toggle GPIO or reschedule.

Hardware Timer API & Usage

One-shot hardware timer

```
static struct tiku_htimer ht;

void my_isr_callback(struct tiku_htimer *t, void *ptr) {
    /* Runs in ISR context --- keep short */
    toggle_pin();
}

/* Schedule 100ms from now */
tiku_htimer_set(&ht, TIKU_HTIMER_NOW() + TIKU_HTIMER_SECOND / 10,
                my_isr_callback, NULL);
```

Periodic hardware timer (self-rescheduling)

```
#define PERIOD (TIKU_HTIMER_SECOND / 10)  /* 100 ms */

void periodic_isr(struct tiku_htimer *t, void *ptr) {
    toggle_pin();
    /* Drift-free: reschedule from original scheduled time */
    tiku_htimer_set(t, TIKU_HTIMER_TIME(t) + PERIOD, periodic_isr, NULL);
}

/* Start first tick */
tiku_htimer_set(&ht, TIKU_HTIMER_NOW() + PERIOD, periodic_isr, NULL);
```

Hardware Timer Configuration Presets

Preset	Source	Freq	Resolution	Notes
High Accuracy	SMCLK /8	1 MHz	1 μs	Default; precise timing
Balanced	SMCLK /64	125 kHz	8 μs	Good balance
Low Power	ACLK	10 kHz	100 μs	Crystal-dependent
Ultra Low Power	ACLK (VLO)	~10 kHz	~100 μs	VLO varies 4–20 kHz
Custom	User-defined	—	—	Set source, dividers

arch/msp430/tiku_htimer_config.h — selecting a preset

```
/* Uncomment exactly one: */
#define TIKU_HTIMER_CONFIG_HIGH_ACCURACY     /* 1 MHz (default) */
// #define TIKU_HTIMER_CONFIG_BALANCED        /* 125 kHz */
// #define TIKU_HTIMER_CONFIG_LOW_POWER       /* ~10 kHz */
// #define TIKU_HTIMER_CONFIG_CUSTOM          /* user-defined */
```

Custom configuration example

```
#define TIKU_HTIMER_CONFIG_CUSTOM
#define TIKU_HTIMER_CLOCK_SOURCE  TIKU_HTIMER_SOURCE_SMCLK
#define TIKU_HTIMER_DIVIDER       TIKU_HTIMER_DIV_4
#define TIKU_HTIMER_EX_DIVIDER    TIKU_HTIMER_EXDIV_4
#define TIKU_HTIMER_BASE_FREQ     8000000UL   /* -> 500 kHz */
```

Hardware Timer ISR Internals

arch/msp430/tiku_htimer_arch.c

```c
/* Timer A1 ISR */
TIKU_ISR(TIMER1_A0_VECTOR,
         tiku_htimer_isr)
{
    tiku_htimer_run_next();
}

/* Schedule compare match */
void tiku_htimer_arch_schedule(
    tiku_htimer_clock_t t)
{
    TA1CCR0 = t;
    TA1CCTL0 &= ~CCIFG;
    TA1CCTL0 |= CCIE;
}

/* Read counter (double-read) */
tiku_htimer_clock_t
tiku_htimer_arch_now(void)
{
    tiku_htimer_clock_t t1, t2;
    do { t1 = TA1R; t2 = TA1R; }
    while (t1 != t2);
```

kernel/timers/tiku_htimer.c

```c
static struct tiku_htimer
    *pending = NULL;

void tiku_htimer_run_next(void)
{
    if (pending == NULL) return;

    struct tiku_htimer *t = pending;
    pending = NULL;

    /* Callback may reschedule */
    t->func(t, t->ptr);

    /* If callback rescheduled,
       arm the hardware */
    if (pending != NULL) {
        tiku_htimer_arch_schedule(
            pending->time);
    }
}
```

Singleton pattern: pending holds at most one

Pattern: Periodic LED Blink

examples/01_blink/blink.c

```
static struct tiku_timer blink_timer;

TIKU_PROCESS(blink_process, "Blink");

TIKU_PROCESS_THREAD(blink_process, ev, data) {
    TIKU_PROCESS_BEGIN();
    tiku_common_led1_init();
    tiku_timer_set_event(&blink_timer, TIKU_CLOCK_SECOND);

    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);
        tiku_common_led1_toggle();
        tiku_timer_reset(&blink_timer);   /* drift-free */
    }
    TIKU_PROCESS_END();
}
TIKU_AUTOSTART_PROCESSES(&blink_process);
```

Pattern: Dual Callback Timers

examples/06_callback_timer/callback_timer.c (simplified)

```c
static struct tiku_timer timer_a, timer_b;

static void on_timer_a(void *ptr) {
    tiku_common_led1_toggle();
    tiku_timer_reset(&timer_a);              /* 500 ms periodic */
}

static void on_timer_b(void *ptr) {
    tiku_common_led2_toggle();
    tiku_timer_reset(&timer_b);              /* 1.3 s periodic */
}

TIKU_PROCESS_THREAD(callback_demo, ev, data) {
    TIKU_PROCESS_BEGIN();
    tiku_common_led1_init();
    tiku_common_led2_init();

    tiku_timer_set_callback(&timer_a, TIKU_CLOCK_SECOND / 2,
                            on_timer_a, NULL);
    tiku_timer_set_callback(&timer_b,
                            TIKU_CLOCK_SECOND + TIKU_CLOCK_SECOND * 3 / 10,
                            on_timer_b, NULL);

    while (1) { TIKU_PROCESS_WAIT_EVENT(); }
```

Pattern: Timeout

examples/08_timeout/timeout.c (simplified)

```c
static struct tiku_timer timeout_timer, restart_timer;

TIKU_PROCESS_THREAD(timeout_demo, ev, data) {
    TIKU_PROCESS_BEGIN();
    while (1) {
        /* Start 5-second timeout */
        tiku_timer_set_event(&timeout_timer, TIKU_CLOCK_SECOND * 5);

        /* Wait for button OR timeout --- whichever comes first */
        TIKU_PROCESS_WAIT_EVENT();

        if (ev == EVENT_BUTTON_PRESS) {
            tiku_timer_stop(&timeout_timer);   /* cancel timeout */
            tiku_common_led1_on();             /* success */
        } else if (ev == TIKU_EVENT_TIMER) {
            tiku_common_led2_on();             /* timed out */
        }

        /* Show result for 2 seconds, then loop */
        tiku_timer_set_event(&restart_timer, TIKU_CLOCK_SECOND * 2);
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);
        tiku_common_led1_off();
        tiku_common_led2_off();
    }
```

Drift-Free vs Drifting Reschedule

Correct: no drift

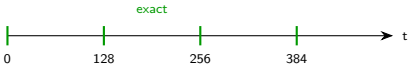
```c
void callback(void *ptr) {
    do_work();
    /* start += interval */
    tiku_timer_reset(&my_timer);
}

/* Ticks: 0, 128, 256, 384, ...
   Exact multiples of interval */
```

Wrong: cumulative drift

```c
void callback(void *ptr) {
    do_work();  /* takes ~3 ticks */
    /* start = now */
    tiku_timer_restart(&my_timer);
}

/* Ticks: 0, 131, 262, 393, ...
   Drifts by ~3 ticks each cycle */
```



Timer Module Files

Layer	File	Role
Clock	`kernel/timers/tiku_clock.h`	Clock API, types, macros
	`kernel/timers/tiku_clock.c`	Delegates to arch layer
Software Timer	`kernel/timers/tiku_timer.h`	Timer API (event + callback)
	`kernel/timers/tiku_timer.c`	Timer process, linked list
Hardware Timer	`kernel/timers/tiku_htimer.h`	htimer API (ISR-driven)
	`kernel/timers/tiku_htimer.c`	Singleton dispatch
Arch (MSP430)	`arch/.../tiku_timer_arch.h`	Clock config, tick rate
	`arch/.../tiku_timer_arch.c`	Timer A0 ISR, ACLK setup
	`arch/.../tiku_htimer_arch.h`	htimer arch interface
	`arch/.../tiku_htimer_arch.c`	Timer A1 ISR, scheduling
	`arch/.../tiku_htimer_config.h`	Preset configurations
Tests	`tests/test_timer.c`	6 test cases

Test Configuration

tests/tiku_test_config.h

```
#define TEST_ENABLE 1

/* Software timer tests */
#define TEST_TIMER            1
#define TEST_TIMER_EVENT      1
#define TEST_TIMER_CALLBACK   1
#define TEST_TIMER_PERIODIC   1
#define TEST_TIMER_STOP       1

/* Hardware timer tests */
#define TEST_HTIMER_BASIC     1
#define TEST_HTIMER_PERIODIC 1
```

Test constants:

```
/* 1 second */
#define TEST_TIMER_INTERVAL \
    TIKU_CLOCK_SECOND
/* 250 ms */
#define TEST_TIMER_SHORT \
    (TIKU_CLOCK_SECOND / 4)
/* Periodic repeat count */
#define TEST_TIMER_PERIODIC_CNT 3
/* htimer: 100 ms */
#define TEST_HTIMER_PERIOD \
    (TIKU_HTIMER_SECOND / 10)
/* htimer repeat count */
#define TEST_HTIMER_REPEAT_CNT 5
```

Test Cases Overview

#	Test Flag	What It Verifies
1	`TEST_TIMER_EVENT`	Event timer posts `TIKU_EVENT_TIMER` to process
2	`TEST_TIMER_CALLBACK`	Callback timer fires function; count == 1
3	`TEST_TIMER_PERIODIC`	`tiku_timer_reset()` reschedules 3 times; drift-free
4	`TEST_TIMER_STOP`	`tiku_timer_stop()` cancels before expiration
5	`TEST_HTIMER_BASIC`	One-shot htimer fires ISR callback exactly once
6	`TEST_HTIMER_PERIODIC`	Self-rescheduling htimer fires 5 times; drift-free

Test Deep Dive: Event & Callback

Event timer test

```c
static struct tiku_timer t;

TIKU_PROCESS_THREAD(test_ev, ev, data)
{
    TIKU_PROCESS_BEGIN();

    tiku_timer_set_event(&t,
        TEST_TIMER_INTERVAL);

    TIKU_PROCESS_WAIT_EVENT_UNTIL(
        ev == TIKU_EVENT_TIMER);

    TEST_ASSERT(
        tiku_timer_expired(&t));
    TEST_PASS("event timer OK");

    TIKU_PROCESS_END();
}
```

Callback timer test

```c
static struct tiku_timer t;
static volatile int cb_count = 0;

void test_cb(void *ptr) {
    cb_count++;
}

TIKU_PROCESS_THREAD(test_cb_proc,
    ev, data)
{
    TIKU_PROCESS_BEGIN();

    cb_count = 0;
    tiku_timer_set_callback(&t,
        TEST_TIMER_SHORT,
        test_cb, NULL);

    /* Wait past expiration */
    tiku_clock_wait(
        TEST_TIMER_SHORT * 2);

    TEST_ASSERT(cb_count == 1);
    TEST_PASS("callback OK");
```

Test Deep Dive: Periodic & Stop

Periodic callback test

```c
static volatile int count = 0;

void periodic_cb(void *ptr) {
    count++;
    if (count < TEST_TIMER_PERIODIC_CNT)
        tiku_timer_reset(&t);
    /* else: don't reschedule */
}

/* After sufficient time: */
TEST_ASSERT(count == 3);
TEST_ASSERT(tiku_timer_expired(&t));
TEST_PASS("periodic OK");
```

Verifies:

- Fires exactly 3 times
- `reset()` reschedules drift-free
- Timer expires when not rescheduled

Timer stop test

```c
static volatile int fired = 0;

void stop_cb(void *ptr) {
    fired++;
}

/* Set 1-second callback */
tiku_timer_set_callback(&t,
    TEST_TIMER_INTERVAL,
    stop_cb, NULL);

/* Stop before expiration */
tiku_timer_stop(&t);

/* Wait past original expiration */
tiku_clock_wait(
    TEST_TIMER_INTERVAL * 2);

TEST_ASSERT(fired == 0);
TEST_PASS("stop OK");
```

Verifies callback **never fires** after `stop()`.

Summary

Three Timer Layers

- **System Clock**: 128 Hz tick from 32.768 kHz crystal
- **Software Timers**: event + callback modes, linked list, drift-free reset
- **Hardware Timer**: ISR-driven, 1 μs resolution, singleton

Key Design Choices

- Wraparound-safe 16-bit arithmetic
- Double-read for atomic counter access
- Timer process auto-cleans dead process timers
- Static allocation throughout

Configuration

- Tick rate: power-of-2 (default 128)
- htimer presets: high accuracy, balanced, low power, custom
- Per-subsystem debug flags

Test Coverage

6 tests: event timer, callback timer, periodic, stop, htimer one-shot, htimer periodic.

Thank You

TikuOS Timer Subsystem

Clock, Software Timers & Hardware Timers for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
Clock:	`kernel/timers/tiku_clock.h`
Timers:	`kernel/timers/tiku_timer.h`
htimer:	`kernel/timers/tiku_htimer.h`
Tests:	`tests/test_timer.c`